MT5 Futures Hedging Dashboard

Technical Reference & Architecture Book

A deep-dive into the Flask backend, SQLite data layer, single-page JavaScript frontend, MetaTrader 5 desktop connector, and production deployment strategy powering a multi-tier proprietary trading management platform.

Version 1.2.0 · March 01, 2026
github.com/Oukaharry/FuturesAutomatedFeed
Author: Oukaharry · Branch: main

TABLE OF CONTENTS

CHAPTER 1: SYSTEM OVERVIEW

1.1 What This System Does

The **MT5 Futures Hedging Dashboard** is a full-stack web application that aggregates, tracks, and visualises the proprietary trading activities of multiple traders across several prop-firm evaluation pipelines. It pulls live data from MetaTrader 5 trading accounts, stores it in a structured database, and exposes that data through a role-based web dashboard and a secured REST API.

The system supports a four-tier hierarchy: **Super Admin** → **Admin** → **Trader** → **Client**. Each tier has progressively scoped access rights enforced at both the route level (via decorator guards) and the database query level.

1.2 Architecture Diagram

Layer	Technology	Role
Desktop App	Python + MetaTrader5 API	Extracts live MT5 data, pushes to dashboard via API
Web Backend	Flask 3 + Gunicorn	REST API, session management, role-based access control
Database	SQLite (dev) / PostgreSQL (prod)	Persistent storage of client data, audit log, history
Frontend	Vanilla JS + CSS3	Single-page dashboard, inline editing, infinite scroll
Auth	PBKDF2-SHA256 sessions + API keys	Cookie sessions for UI; X-API-Key header for apps
Deployment	Ubuntu VPS + Nginx + Gunicorn	Reverse-proxy, SSL termination, process management

1.3 User Roles & Permissions

Role	Login Method	Can See	Can Edit
super_admin	Password (PBKDF2)	All admins, traders, clients	Everything
admin	Password (PBKDF2)	Own traders + clients	Trader/client data
trader	Password (PBKDF2)	Own clients only	Client evaluations
client	Email + Password	Own data only	Prop Day notes only
API (Trader App)	X-API-Key header	Scoped to trader's clients	Push MT5 data

1.4 Data Flow

1. The **Trader Companion** desktop app connects to a live MT5 terminal and extracts deals, positions, account balances, and evaluation states at a configurable interval (typically every 60 seconds).

2. The data is serialised to JSON and POSTed to /api/update_data on the Flask server using an **X-API-Key** header for authentication.

3. The server merges the incoming payload with existing client data, preserving soft-deleted evaluation rows and dashboard-side edits, then saves to the database and writes a versioned snapshot to the

audit/history tables.

4. The web dashboard polls /api/get_data?client_id=… and renders the data into a rich HTML table with sticky headers, inline editing, and an expandable financials panel.

CHAPTER 2: FLASK BACKEND

2.1 Application Bootstrap

The Flask application lives in **dashboard/app.py** (~4 400 lines). On startup it initialises the database, starts the midnight watermark scheduler, configures rate limiting via **Flask-Limiter**, and sets a cryptographically random secret key for sessions.

■ dashboard/app.py — app initialisation

```
app = Flask(__name__) app.config['SEND_FILE_MAX_AGE_DEFAULT'] = 0 app.secret_key =
os.getenv('FLASK_SECRET_KEY', secrets.token_hex(32)) limiter = Limiter( app=app,
key_func=get_remote_address, default_limits=["10000 per day", "2000 per hour"],
storage_uri="memory://" ) # Start Midnight Watermark Scheduler from dashboard.scheduler import
start_scheduler start_scheduler()
```

2.2 Authentication Decorators

Four decorators guard every route. They are composed using Python's functools.wraps to preserve the original function metadata.

require_api_key

Extracts the X-API-Key header, validates it against the database, injects the resolved request.api_user context, and logs the access.

■ dashboard/app.py — require_api_key decorator

```
def require_api_key(f): @wraps(f) def decorated_function(*args, **kwargs): api_key =
request.headers.get('X-API-Key') client_ip = get_remote_address() if not api_key:
log_action('API_ACCESS_DENIED', 'unknown', 'no_key', client_ip, 'Missing API key', False)
return jsonify({"status": "error", "message": "API key required"}), 401 user_info =
validate_api_key(api_key) if not user_info: log_action('API_ACCESS_DENIED', 'unknown',
api_key[:12], client_ip, 'Invalid API key', False) return jsonify({"status": "error",
"message": "Invalid API key"}), 403 request.api_user = user_info log_action('API_ACCESS',
'trader', user_info.get('trader', 'unknown'), client_ip, f"Endpoint: {request.endpoint}")
return f(*args, **kwargs) return decorated_function
```

require_role(*allowed_roles)

Session-cookie based role gate. Reads the session_token cookie, validates it, and checks user_type against a whitelist of allowed roles. Returns HTTP 403 on mismatch.

■ dashboard/app.py — require_role decorator

```
def require_role(*allowed_roles): def decorator(f): @wraps(f) def decorated_function(*args,
**kwargs): session_token = request.cookies.get('session_token') if not session_token: return
jsonify({"status": "error", "message": "Authentication required"}), 401 session_info =
validate_session(session_token) if not session_info: return jsonify({"status": "error",
"message": "Invalid or expired session"}), 401 user_type = session_info.get('user_type') if
user_type not in allowed_roles: log_action('ACCESS_DENIED', user_type,
session_info.get('user_identifier'), get_remote_address(), f"Required roles: {allowed_roles}",
False) return jsonify({"status": "error", "message": "Access denied."}), 403
request.session_user = session_info return f(*args, **kwargs) return decorated_function return
decorator
```

2.3 Key API Endpoints

Method	Endpoint	Auth	Description
GET	/api/get_data	Session / API Key	Fetch client JSON data
POST	/api/update_data	Session / API Key	Push new MT5 snapshot
POST	/api/login	None (public)	Username+password login, returns session cookie
POST	/api/auth/login	None (public)	Unified multi-role login endpoint
POST	/api/rollback	Session	Rollback client data to a previous version
POST	/api/notes	Session	Save / delete inline cell notes
GET	/api/history	Session	List version history for a client
POST	/api/generate_key	Admin Password	Generate a new API key for a trader
GET	/api/audit_log	Session (admin+)	Return recent audit log entries

2.4 /api/get_data — Data Retrieval

This endpoint serves as the primary data source for the dashboard. It accepts both session cookies (for browser clients) and API-key headers (for the trader companion app), resolves the caller's identity, checks access permissions with can_access_client(), and enriches the raw JSON with inline notes and computed historical MT5 totals before returning the response.

■ dashboard/app.py — get_data() (condensed)

```
@app.route('/api/get_data') def get_data(): session_token =
request.cookies.get('session_token') api_key = request.headers.get('X-API-Key') # --- resolve
caller identity --- if session_token: session_info = validate_session(session_token) user_type
= session_info['user_type'] user_identifier = session_info['user_identifier'] elif api_key:
key_info = validate_api_key(api_key) user_type = 'api' user_identifier = key_info['owner']
else: return jsonify({"status": "error", "message": "Auth required"}), 401 client_id =
request.args.get('client_id') if not can_access_client(user_type, user_identifier, client_id):
return jsonify({"status": "error", "message": "Access denied"}), 403 data =
get_client_data(client_id) # Inject visual notes notes = get_client_notes(client_id) for i, ev
in enumerate(data.get('evaluations', [])): if i in notes: ev['_notes'] = notes[i]
data['status'] = 'success' return jsonify(data)
```

2.5 /api/update_data — Dual-Auth Merge

The update endpoint performs a server-side merge strategy: incoming data is layered on top of existing data so that missing keys (e.g. the dashboard user did not send deals) default to their current stored values. A versioned snapshot is written to the history table after every successful save.

■ dashboard/app.py — update_data() core merge logic

```
@app.route('/api/update_data', methods=['POST']) @limiter.limit("60 per minute") def
update_data(): data = request.json existing_data = get_client_data(client_id) or {} evaluations
= normalize_evaluations( data.get('evaluations', existing_data.get('evaluations', [])) )
client_data = { 'deals': data.get('deals', existing_data.get('deals', [])), 'positions':
data.get('positions', existing_data.get('positions', [])), 'account': data.get('account',
existing_data.get('account', {})), 'evaluations': evaluations, 'statistics':
data.get('statistics', existing_data.get('statistics', {})), # ... other fields merged
similarly } success, version = save_client_data_with_history( client_id, client_data,
action='UPDATE', changed_by=user_identifier, changed_by_type=user_type,
ip_address=get_remote_address(), change_source='dashboard_edit', change_description=f'Manual
edit by {user_type}' ) return jsonify({"status": "success", "version": version})
```

2.6 Route Structure

Routes are organised into logical groups. All routes serving HTML pages are decorated with @require_session for redirect-to-login behaviour, while JSON API routes return structured error objects.

Route Pattern	Template	Access
/	login.html	Public
/super_admin	super_admin.html	super_admin only
/admin/	admin_dashboard.html	admin / super_admin
/trader/	index.html	trader / admin / super_admin
/dashboard/	index.html	client (own data only)
/financial_overview	financial_overview.html	super_admin only

2.7 Account Signature Matching

MT5 deal comments often contain truncated account numbers (e.g. FNFT...59574). The server resolves these to evaluation rows using a signature-based algorithm that extracts the first-4 and last-4/5 digits normalised to lowercase.

■ dashboard/app.py — get_account_signature()

```
def get_account_signature(account_number): account_str = str(account_number).strip() # Truncated format: PREFIX...SUFFIX e.g. FNFT...59574 if '...' in account_str: parts = account_str.split('...') prefix = parts[0][:4] if len(parts[0]) >= 4 else parts[0] suffix = parts[1] # keep full suffix return (prefix + suffix).lower() # Standard: first 4 + last 4 if len(account_str) < 8: return account_str.lower() return (account_str[:4] + account_str[-4:]).lower()
```

CHAPTER 3: DATABASE LAYER

3.1 Engine & Connection Strategy

The database module (**dashboard/database.py**, ~1 300 lines) uses Python's standard sqlite3 library in development, with a thin abstraction layer that can be swapped for psycopg2 (PostgreSQL) in production by changing a single environment variable.

All connections are obtained via a context manager (get_connection()) that sets row_factory = sqlite3.Row for named-column access and enables WAL journal mode for better concurrent read performance.

3.2 Schema Reference

user_credentials

■ dashboard/database.py — user_credentials table DDL

```
CREATE TABLE IF NOT EXISTS user_credentials ( id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT NOT NULL, email TEXT, password_hash TEXT NOT NULL, salt TEXT NOT NULL, user_type TEXT NOT NULL, -- super_admin | admin | trader | client parent_admin TEXT, -- hierarchy link parent_trader TEXT, -- hierarchy link is_active INTEGER DEFAULT 1, must_change_password INTEGER DEFAULT 0, last_login TEXT, created_at TEXT DEFAULT (datetime('now')), updated_at TEXT DEFAULT (datetime('now')), UNIQUE(username, user_type) )
```

clients_data

■ dashboard/database.py — clients_data table DDL

```
CREATE TABLE IF NOT EXISTS clients_data ( id INTEGER PRIMARY KEY AUTOINCREMENT, client_id TEXT NOT NULL UNIQUE, deals TEXT DEFAULT '[]', -- JSON array positions TEXT DEFAULT '[]', -- JSON array account TEXT DEFAULT '{}', -- JSON object evaluations TEXT DEFAULT '[]', -- JSON array (main eval sheet rows) updated_at TEXT DEFAULT (datetime('now')) )
```

audit_log

■ dashboard/database.py — audit_log table DDL

```
CREATE TABLE IF NOT EXISTS audit_log ( id INTEGER PRIMARY KEY AUTOINCREMENT, timestamp TEXT NOT NULL, action TEXT NOT NULL, -- e.g. 'DATA_UPDATE', 'LOGIN', 'ROLLBACK' user_type TEXT, user_identifier TEXT, ip_address TEXT, description TEXT, success INTEGER DEFAULT 1, client_id TEXT )
```

data_history (version snapshots)

■ dashboard/database.py — data_history table DDL

```
CREATE TABLE IF NOT EXISTS data_history ( id INTEGER PRIMARY KEY AUTOINCREMENT, client_id TEXT NOT NULL, version INTEGER NOT NULL, data_snapshot TEXT NOT NULL, -- full JSON snapshot action TEXT, changed_by TEXT, changed_by_type TEXT, ip_address TEXT, change_source TEXT, change_description TEXT, created_at TEXT DEFAULT (datetime('now')) )
```

daily_watermarks & waterlog_periods

■ dashboard/database.py — watermark tables DDL

```
CREATE TABLE IF NOT EXISTS daily_watermarks ( id INTEGER PRIMARY KEY AUTOINCREMENT, client_id TEXT NOT NULL, eval_index INTEGER NOT NULL, date TEXT NOT NULL, balance REAL, drawdown_pct REAL, created_at TEXT DEFAULT (datetime('now')), UNIQUE(client_id, eval_index, date) ); CREATE TABLE IF NOT EXISTS waterlog_periods ( id INTEGER PRIMARY KEY AUTOINCREMENT, client_id TEXT NOT NULL, eval_index INTEGER NOT NULL, start_date TEXT NOT NULL, end_date TEXT, peak_balance REAL,
```

```
trough_balance REAL, max_drawdown_pct REAL, status TEXT DEFAULT 'open' );
```

3.3 Password Security

Passwords are hashed using PBKDF2-HMAC-SHA256 with 100 000 iterations and a cryptographically random 32-byte salt generated by secrets.token_hex(32). Comparison uses secrets.compare_digest() to prevent timing attacks.

■ dashboard/database.py — hash_password / verify_password

```
import secrets, hashlib def hash_password(password: str, salt: str = None) -> tuple: if salt is None: salt = secrets.token_hex(32) # 64-char hex string password_hash = hashlib.pbkdf2_hmac( 'sha256', password.encode('utf-8'), salt.encode('utf-8'), 100_000 # OWASP-recommended iteration count ).hex() return password_hash, salt def verify_password(password: str, stored_hash: str, salt: str) -> bool: computed_hash, _ = hash_password(password, salt) return secrets.compare_digest(computed_hash, stored_hash) # constant-time
```

3.4 Login Verification Flow

■ dashboard/database.py — verify_user_password() (condensed)

```
def verify_user_password(username, password, user_type): with get_connection() as conn: cursor = conn.cursor() cursor.execute(''' SELECT id, username, email, password_hash, salt, user_type, parent_admin, parent_trader, must_change_password FROM user_credentials WHERE username = ? AND user_type = ? AND is_active = 1 ''', (username, user_type)) row = cursor.fetchone() if row is None: return None # user not found if not verify_password(password, row['password_hash'], row['salt']): return None # wrong password # Stamp last_login cursor.execute( 'UPDATE user_credentials SET last_login = ? WHERE id = ?', (datetime.now().isoformat(), row['id']) ) conn.commit() return { 'id': row['id'], 'username': row['username'], 'email': row['email'], 'user_type': row['user_type'], 'parent_admin': row['parent_admin'], 'parent_trader': row['parent_trader'], 'must_change_password': bool(row['must_change_password']) }
```

3.5 Versioned History & Rollback

Every data mutation triggers save_client_data_with_history(), which writes a full JSON snapshot to data_history with an incrementing version counter. The dashboard's version history panel lists these snapshots, and any version can be restored via /api/rollback.

■ Rollback is non-destructive: the restored snapshot itself becomes version N+1, so the full history is always preserved and any rollback can itself be rolled back.

3.6 Soft Delete Pattern

Evaluation rows deleted from the dashboard are marked with a _deleted: true flag in the JSON array rather than being physically removed. This means concurrent pushes from the Trader Companion app cannot accidentally resurrect deleted rows.

■ dashboard/templates/index.html — soft delete (JS)

```
// Mark as soft-deleted instead of splicing the array currentData.evaluations[index]._deleted = true; // Render loop skips deleted rows rowsToRender = currentData.evaluations.filter( ev => !ev._deleted );
```

CHAPTER 4: FRONTEND APPLICATION

4.1 Architecture Choice

The entire UI is delivered as a single HTML file — **dashboard/templates/index.html** (~4 300 lines). No JavaScript framework or build tool is used; all interactivity is implemented in vanilla ES6+ code embedded directly in the template. This choice eliminates build complexity and makes the app trivially deployable as a Jinja2 template served by Flask.

4.2 Layout Structure

DOM Element	CSS Class / ID	Role
	.header-title-bar + .header-controls-bar	Sticky two-row header: logo/title + tabs/controls
	#main-content	Scrollable content area below the header
	#eval-table	Main evaluations data grid (~60+ columns)
	—	Three sticky header rows: group labels, column names, filters
	#eval-tbody	Rendered evaluation rows (paginated, 50/page)
	#eval-load-more-bar	'Show More' pagination trigger bar
	#section-sticky-label	Floating pill inside EVAL INFO header showing current scroll section

4.3 Sticky Header System

The table uses a three-tier sticky-header system implemented entirely in CSS. Each tier is positioned with a different top offset so all three rows remain visible when the user scrolls down through rows.

■ dashboard/static/css/style.css — sticky header tiers

```
/* Row 1: Section group labels (EVAL INFO, EVAL PHASE, etc.) */ thead tr:nth-child(1) th {
position: sticky; top: 0; z-index: 3000; } /* Row 2: Column names */ thead tr:nth-child(2) th {
position: sticky; top: 42px; /* height of row 1 */ z-index: 2000; } /* Row 3: Column filter
inputs */ thead tr:nth-child(3) th { position: sticky; top: 84px; /* row 1 + row 2 heights */
z-index: 1500; } /* Full-page header also sticky */ header { position: sticky; top: 0; z-index:
5000; flex-direction: column; }
```

4.4 Scroll Section Indicator (Pill Badge)

A coloured pill badge inside the EVAL INFO group header updates in real-time as the user scrolls the table horizontally, showing which logical section is currently in view (e.g. EVAL INFO, EVAL PHASE, FUNDED PHASE, FARMING).

■ dashboard/templates/index.html — updateScrollIndicator()

```
const sectionLabelMap = { 'eval-info': { label: 'EVAL INFO', color: '#60a5fa' }, // blue
'eval-phase': { label: 'EVAL PHASE', color: '#4ade80' }, // green 'funded-phase':{ label:
'FUNDED PHASE',color: '#fb7185' }, // pink-red 'farming': { label: 'FARMING', color: '#c084fc'
}, // purple }; function updateScrollIndicator() { const pill =
document.getElementById('section-sticky-label'); if (!pill) return; const tableRect =
evalTable.getBoundingClientRect(); const viewCenter = tableRect.left + tableRect.width / 2; //
```

```
Find which group header overlaps the centre of the viewport for (const [key, config] of
Object.entries(sectionLabelMap)) { const header =
document.querySelector(`th[data-section="${key}"]`); if (!header) continue; const r =
header.getBoundingClientRect(); if (r.left <= viewCenter && r.right >= viewCenter) {
pill.textContent = config.label; pill.style.background = config.color + '33'; // 20% opacity
fill pill.style.color = config.color; pill.style.borderColor= config.color; break; } } } //
Attach to both scroll axes evalTable.closest('.table-wrapper') .addEventListener('scroll',
updateScrollIndicator, { passive: true });
```

4.5 Pagination Strategy

Evaluation rows are rendered in pages of 50 to avoid DOM congestion on accounts with hundreds of evaluations. A sticky 'Show More' bar at the bottom of the table loads the next batch without any network request — all data is already resident in the JavaScript currentData object.

■ dashboard/templates/index.html — pagination globals

```
let evalPage = 1; const EVAL_PAGE_SIZE = 50; function renderEvaluationsTable(data, append =
false) { if (!append) evalPage = 1; const all = (data.evaluations || []) .filter(ev =>
!ev._deleted) .sort((a, b) => b.originalIndex - a.originalIndex); // newest first const slice =
all.slice(0, evalPage * EVAL_PAGE_SIZE); // ... build and insert table rows ... // Show / hide
the 'load more' bar const bar = document.getElementById('eval-load-more-bar'); const more =
all.length > slice.length; bar.style.display = more ? 'flex' : 'none'; if (more) {
bar.querySelector('.load-more-count').textContent = `${all.length - slice.length} more`; } }
function loadMoreEvaluations() { evalPage++; renderEvaluationsTable(currentData, true); }
```

4.6 Inline Editing & Save Flow

Each cell that can be edited triggers a saveData() call which PATCHes the entire current state back to the server. The function throttles saves to avoid hammering the API on rapid keystrokes using a debounce timer.

■ dashboard/templates/index.html — saveData() (condensed)

```
let saveTimer = null; function saveData(changeDescription = 'Cell edit') {
clearTimeout(saveTimer); saveTimer = setTimeout(async () => { const payload = { ...currentData,
client_id: currentClientId, action_type: 'UPDATE', change_description: changeDescription };
const resp = await fetch('/api/update_data', { method: 'POST', headers: { 'Content-Type':
'application/json' }, body: JSON.stringify(payload) }); const result = await resp.json(); if
(result.status === 'success') { showToast('Saved ✓', 'success'); currentVersion =
result.version; } else { showToast('Save failed: ' + result.message, 'error'); } }, 600); // 600
ms debounce }
```

4.7 Brand Colour System

Each prop firm has a distinct brand accent colour applied to evaluation rows and UI indicators, making it instantly clear which firm is associated with each account row.

Prop Firm	Brand Colour	Hex	Applied To
Topstep	Yellow / Gold	#fcd34d	Row accent, badges, indicators
Lucid	Black / Slate	#334155	Row accent, muted tones
Alpha	Green	#4ade80	Row accent, profit indicators
FTMO	Blue	#60a5fa	Row accent
My Forex Funds	Orange	#fb923c	Row accent

CHAPTER 5: TRADER COMPANION DESKTOP APP

5.1 Purpose

The **Trader Companion** is a Windows-native Python application built with Tkinter that runs continuously alongside a MetaTrader 5 terminal. It uses the official MetaTrader5 Python package to fetch live trading data and push it to the dashboard server every 60 seconds.

5.2 MT5 Data Extraction

■ `trader_companion/trader_app.py — live data fetch (pattern)`

```
import MetaTrader5 as mt5 def fetch_mt5_data(self): if not mt5.initialize(): self.log('MT5 init failed') return None # Account summary account_info = mt5.account_info()._asdict() # All historical deals (closed trades) deals = mt5.history_deals_get( datetime(2020, 1, 1), datetime.now() ) deals_list = [d._asdict() for d in deals] if deals else [] # Open positions positions = mt5.positions_get() positions_list = [p._asdict() for p in positions] if positions else [] mt5.shutdown() return { 'account': account_info, 'deals': deals_list, 'positions': positions_list }
```

5.3 Evaluation Matching Algorithm

The most complex part of the Trader Companion is matching MT5 deal comments (which contain truncated account numbers) to rows in the user's evaluation spreadsheet. The matching uses a multi-strategy fallback: exact match → signature match (first4+last4 digits) → last-N-digits match.

Strategy	Input	Match Condition
1. Exact	Full account string	account_str == eval['Account #']
2. Signature	First4 + Last4 digits	signature(deal) == signature(eval['Account #'])
3. Last-N	Last 5 digits	deal[-5:] == eval['Account #'][-5:]
4. Prefix	First 4 chars	Used only for firm-specific disambiguation

5.4 Push Protocol

After processing, the app sends a POST request to /api/update_data with the full payload including deals, positions, and updated evaluations. Regular pushes send evaluations: [], preserving server-side edits. Hedging sync pushes first fetch the current evaluations, update the hedge-related fields, then push the merged result.

■ `trader_companion/trader_app.py — push call (pattern)`

```
def push_to_server(self, data: dict, include_evaluations: bool = False): if not include_evaluations: data['evaluations'] = [] # preserve server-side eval edits response = requests.post( f"{self.server_url}/api/update_data", json=data, headers={'X-API-Key': self.api_key}, timeout=30 ) result = response.json() if result.get('status') == 'success': self.log(f"Pushed. Server version: {result.get('version')}") else: self.log(f"Push failed: {result.get('message')}")
```

5.5 Desktop Build (PyInstaller)

The app is distributed as a standalone .exe using PyInstaller. The spec file bundles all Python dependencies, the Tkinter runtime, and any required assets into a single-file executable that end users can run on Windows without installing Python or any packages.

Spec File	Version	Entry Point
Trader_Companion_v1.2.0.spec	1.2.0	trader_companion/trader_app.py
BallerQuotes_Trader_Companion_v1.2.0.spec	1.2.0	trader_companion/baller_app.py

■ Build command

```
# From workspace root, with PyInstaller installed: pyinstaller Trader_Companion_v1.2.0.spec
--clean # Output: dist/Trader_Companion_v1.2.0/Trader_Companion_v1.2.0.exe
```

CHAPTER 6: FINANCIAL CALCULATIONS MODULE

6.1 Overview

dashboard/financial_overview.py is the analytics engine. It processes raw deal/position data from the database and produces aggregated financial metrics: running P&L;, deposit growth, fee curves, hedge efficiency, and per-trader performance statistics.

6.2 Core Functions

Function	Output
calculate_propfirm_overview()	Aggregated P&L;, deposits, withdrawals per prop firm
get_payouts_history()	Chronological list of payout events
get_portfolio_growth_data()	Time-series of combined portfolio value
get_cumulative_deposits()	Running deposit total over time
get_cumulative_trading_profit()	Running realised profit over time
get_cumulative_fees_data()	Running fee expenditure over time
get_cumulative_hedge_data()	Running hedge result vs. sheet value
calculate_trader_stats()	Per-trader win rate, avg profit, drawdown stats
get_client_performance_stats()	Per-client performance breakdown
get_cached_clients_dataset()	Memoised full dataset across all clients

6.3 Watermark Scheduler

A background thread (**dashboard/scheduler.py**) wakes at midnight UTC every day, iterates over all active evaluations, and records a **daily watermark** — the end-of-day balance and maximum drawdown — to the daily_watermarks table. This enables accurate drawdown tracking across multi-day sessions.

■ Phase transitions (e.g. Challenge passed → Funded) are detected automatically by comparing the current balance watermark against phase-definition thresholds stored in the phase_definitions table.

CHAPTER 7: DEPLOYMENT & OPERATIONS

7.1 Production Stack

Component	Technology	Detail
OS	Ubuntu 22.04 LTS	VPS or dedicated server
WSGI Server	Gunicorn 21+	4 workers, gevent worker class
Proxy	Nginx	SSL termination, static files, rate limiting
SSL	Let's Encrypt / Certbot	Auto-renewal via systemd timer
DB (prod)	PostgreSQL 15	Managed instance; SQLite used for dev
Process Mgmt	systemd	Auto-restart on crash, boot-time start
Secrets	Environment vars	FLASK_SECRET_KEY, DATABASE_URL, etc.

7.2 Gunicorn Configuration

■ gunicorn.conf.py

```
workers = 4 worker_class = 'gevent' bind = '127.0.0.1:5000' timeout = 120 keepalive = 5
max_requests = 1000 max_requests_jitter = 50 accesslog = 'logs/access.log' errorlog =
'logs/error.log' loglevel = 'info'
```

7.3 Nginx Reverse Proxy (sample)

■ nginx site configuration (condensed)

```
server { listen 443 ssl; server_name yourdomain.com; ssl_certificate
/etc/letsencrypt/live/yourdomain.com/fullchain.pem; ssl_certificate_key
/etc/letsencrypt/live/yourdomain.com/privkey.pem; location / { proxy_pass
http://127.0.0.1:5000; proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto
$scheme; proxy_read_timeout 120s; } location /static/ { alias /var/www/dashboard/static/;
expires 7d; add_header Cache-Control "public, immutable"; } } # HTTP -> HTTPS redirect server {
listen 80; server_name yourdomain.com; return 301 https://$host$request_uri; }
```

7.4 Environment Variables

Variable	Required	Description
FLASK_SECRET_KEY	Yes	Random 64-char hex string for session signing
DATABASE_URL	Prod	PostgreSQL connection string (postgres://…)
FLASK_ENV	No	'production' disables debug mode
RATE_LIMIT_STORAGE	No	Redis URI for distributed rate limiting
LOG_LEVEL	No	'DEBUG' | 'INFO' | 'WARNING'

7.5 Deployment Script

■ deploy.sh — production deploy (condensed)

```
#!/bin/bash set -e echo "[1/5] Pulling latest code..." git pull origin main echo "[2/5] Installing Python dependencies..." pip install -r requirements-production.txt echo "[3/5] Running database migrations..." python migrations.py echo "[4/5] Collecting static files..." # (No build step needed — static assets served directly by Nginx) echo "[5/5] Reloading Gunicorn..." systemctl reload gunicorn echo "Deploy complete."
```

7.6 Rate Limiting Strategy

Flask-Limiter applies layered rate limits to prevent abuse. API push endpoints allow higher throughput for the Trader Companion app, while authentication endpoints are tightly restricted to block brute-force attacks.

Endpoint Group	Limit	Rationale
/api/auth/login	5/minute	Prevent credential stuffing
/api/update_data	60/minute	Allow frequent MT5 sync pushes
/api/get_data	120/minute	Dashboard polling
Default (all others)	10 000/day, 2 000/hour	General protection

CHAPTER 8: SECURITY ARCHITECTURE

8.1 Defence in Depth

Security is implemented at multiple layers: transport (TLS), network (Nginx rate limiting + firewall), application (RBAC decorators + brute-force lockout), and data (PBKDF2 password hashing + parameterised SQL queries).

8.2 Brute-Force Protection

The login_attempts table records every failed login. After 5 consecutive failures from the same IP or for the same username, the account is locked for 15 minutes. The lockout is checked via is_account_locked() before any credential verification occurs.

■ dashboard/database.py — login_attempts table

```
CREATE TABLE IF NOT EXISTS login_attempts ( id INTEGER PRIMARY KEY AUTOINCREMENT, identifier TEXT NOT NULL, -- username or IP attempt_time TEXT NOT NULL, success INTEGER DEFAULT 0 );
```

8.3 SQL Injection Prevention

All database queries use parameterised statements (cursor.execute(sql, (param,))). No string formatting or concatenation is used for SQL construction anywhere in the codebase.

■ dashboard/database.py — parameterised query (example)

```
# CORRECT — parameterised cursor.execute( 'SELECT * FROM user_credentials WHERE username = ? AND user_type = ?', (username, user_type) ) # NEVER done — vulnerable to injection # cursor.execute(f"SELECT * FROM user_credentials WHERE username = '{username}'")
```

8.4 Session Management

Sessions are stored in the database as a sessions table. Each session token is a 64-character hex string generated by secrets.token_hex(32). Sessions expire after 24 hours of inactivity. Tokens are transmitted exclusively via HttpOnly cookies to prevent JavaScript access.

8.5 Audit Log

Every significant action — login, data update, rollback, access denial, API key generation — is written to the audit_log table with the timestamp, action type, user identity, IP address, and success flag. Super admins can query the full audit log via /api/audit_log.

CHAPTER 9: DEVELOPER WORKFLOWS

9.1 Local Development Setup

■ **Setup commands (Windows PowerShell)**

```
# Clone repository git clone https://github.com/Oukaharry/FuturesAutomatedFeed.git cd
FuturesAutomatedFeed # Create virtual environment python -m venv .venv
.venv\Scripts\Activate.ps1 # Install dependencies pip install -r requirements.txt # Initialise
database python -c "from dashboard.database import init_database; init_database()" # Run
development server python wsgi.py # → http://localhost:5000
```

9.2 Common Debug Scripts

Script	Purpose
debug_fetch.py	Test /api/get_data against a running server
debug_ev.py	Inspect evaluation matching for a specific account
debug_phase_logic.py	Trace phase-transition detection logic
debug_compare.py	Diff two client data JSON snapshots
debug_waterlog.py	Inspect watermark records for a client
check_audit.py	Print recent audit log entries
check_db_table.py	Dump any SQLite table to stdout
reproduce_match.py	Reproduce an account-matching result with test data
quick_test.py	Run lightweight sanity checks against the server

9.3 Git Workflow

■ **Standard commit workflow**

```
git add -A git commit -m "feat: descriptive message" git push origin main # Check remote status
git log --oneline -5 git status
```

9.4 Building the Desktop App

■ **PyInstaller build (from workspace root)**

```
# Ensure PyInstaller is installed pip install pyinstaller # Build using the spec file
pyinstaller Trader_Companion_v1.2.0.spec --clean --noconfirm # Output location #
dist/Trader_Companion_v1.2.0/Trader_Companion_v1.2.0.exe
```

CHAPTER 10: REST API REFERENCE

10.1 Authentication

The API supports two authentication mechanisms: (1) **Session Cookie** — set on login via /api/auth/login and sent automatically by the browser for all subsequent requests; (2) **API Key** — a 32-char hex token sent in the X-API-Key header, intended for the Trader Companion desktop app and programmatic access.

10.2 POST /api/auth/login

■ **Request**

```
POST /api/auth/login Content-Type: application/json { "username": "trader_harry", "password": "s3cur3P@ss", "user_type": "trader" }
```

■ **Response (200 OK)**

```
HTTP/1.1 200 OK Set-Cookie: session_token=abc123...; HttpOnly; SameSite=Strict { "status": "success", "user_type": "trader", "username": "trader_harry", "must_change_password": false }
```

10.3 GET /api/get_data

■ **Request**

```
GET /api/get_data?client_id=harry_client_1 X-API-Key: <your-api-key> # OR session cookie
```

■ **Response schema (200 OK)**

```
{ "status": "success", "deals": [ { "ticket": 123, "symbol": "NQ", "profit": 150.0, ... } ], "positions": [ { "ticket": 456, "symbol": "ES", "volume": 1.0, ... } ], "account": { "balance": 125000, "equity": 124800, "login": 9876543 }, "evaluations":[ { "Firm": "Topstep", "Account #": "TSX123456", ... } ], "statistics": { "hedging_review": { ... }, "account_stats": { ... } }, "last_updated": "2025-01-15T14:32:00" }
```

10.4 POST /api/update_data

■ **Request**

```
POST /api/update_data Content-Type: application/json X-API-Key: <your-api-key> { "identity": { "client": "harry_client_1", "trader": "trader_harry" }, "deals": [ ... ], "positions": [ ... ], "account": { ... }, "evaluations": [], # empty = keep server-side evaluations "statistics": { ... } }
```

■ **Response (200 OK)**

```
{ "status": "success", "message": "Data updated", "version": 42 # new version number in history }
```

10.5 POST /api/rollback

■ **Request**

```
POST /api/rollback Content-Type: application/json Cookie: session_token=... { "client_id": "harry_client_1", "version": 38 # target version to restore }
```

■ **Response (200 OK)**

```
{ "status": "success", "message": "Rolled back to version 38", "new_version": 43 # restoration itself creates a new version }
```

10.6 Error Responses

HTTP Status	When / Meaning
400 Bad Request	Missing required fields in request body
401 Unauthorized	No session cookie and no API key provided
403 Forbidden	Valid credentials but insufficient permissions for resource
404 Not Found	Requested client_id exists but has no saved data yet
429 Too Many Requests	Rate limit exceeded (see per-endpoint limits)
500 Internal Server Error	Unhandled exception — check server log

END OF DOCUMENT

Generated March 01, 2026 · github.com/Oukaharry/FuturesAutomatedFeed